



TechCrush Capstone Project

PROJECT 3: APPLYING CRYPTOGRAPHY IN REAL-WORLD SCENARIOS

Group 6 | Cybersecurity Capstone | June 2026

Focus: encryption, decryption, hashing, public key cryptography, and digital signatures



Edit with WPS Office

Group No. 6 — Active Project 3 Members (1/2)

No.	Member Name	Matric Number
1	John-Francis Nnadi	CYBE/2026/TC-6/0959
2	Dauda Musefiu Adetunji	CYBE/2026/TC-6/0250
3	Oyediji Toluwalase David	CYBE/2026/TC-6/0392
4	Victor Akan Essien	CYBE/2026/TC-6/0042
5	Anikweobi Onyebuchi John	CYBE/2026/TC-6/0163
6	Isaiah Bobby Onyekachi	CYBE/2026/TC-6/0684
7	Andrea Kosisoschukwu Ogbuli	CYBE/2026/TC-6/0960

Only members recorded as active for Project 3 are included.



Edit with WPS Office

Group No. 6 — Active Project 3 Members (2/2)

No.	Member Name	Matric Number
8	Lucky Simeon, Monsigia	CYBE/2026/TC-6/0676
9	Tunde Opeyemi Adejumo	CYBE/2026/TC-6/0872
10	Jamaldeen Akinkunmi Oni	CYBE/2026/TC-6/0893
11	Roqueebah Ogunbowale	CYBE/2026/TC-6/0924
12	UGOCHUKWU PRINCEWILL OKAFOR	CYBE/2026/TC-6/0072
13	OLUWASEUN ADELADAN	CYBE/2026/TC-6/1016

Only members recorded as active for Project 3 are included.



Edit with WPS Office

Outline

Aim and Objective
Methodology
Concept of Project
Tools Used
Practical Implementation
Results and Findings
Conclusion
References

The slides summarise the report and show the practical evidence collected from the cryptography lab.



Edit with WPS Office

Aim and Objective

Aim

To demonstrate how cryptographic techniques are used to protect files, messages, and data integrity in realistic situations.

Main Objectives

Apply symmetric encryption, hashing, public key encryption, and digital signatures to practical cybersecurity tasks.

Project Tasks

Encrypt a file and decrypt it back to readable form. Generate hashes before and after file editing. Use public/private keys for secure messaging. Verify authenticity with a digital signature.

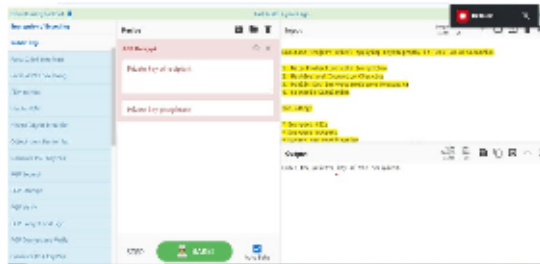


Edit with WPS Office

Methodology



The practical workflow moved from file preparation to secure messaging, integrity checking, and signature verification.



Sample file



Encrypted output



Decrypted result



Edit with WPS Office

Concept of Project

Confidentiality

Only authorised users should be able to read protected data.

Integrity

The content should stay unchanged unless a modification is detected.

Authenticity

The receiver should be able to trust the source of the data.

Project Scenario

A secure file exchange and backup verification workflow: prepare a project file, protect it with encryption, verify its integrity with hashes, and confirm its source using a digital signature.



Edit with WPS Office

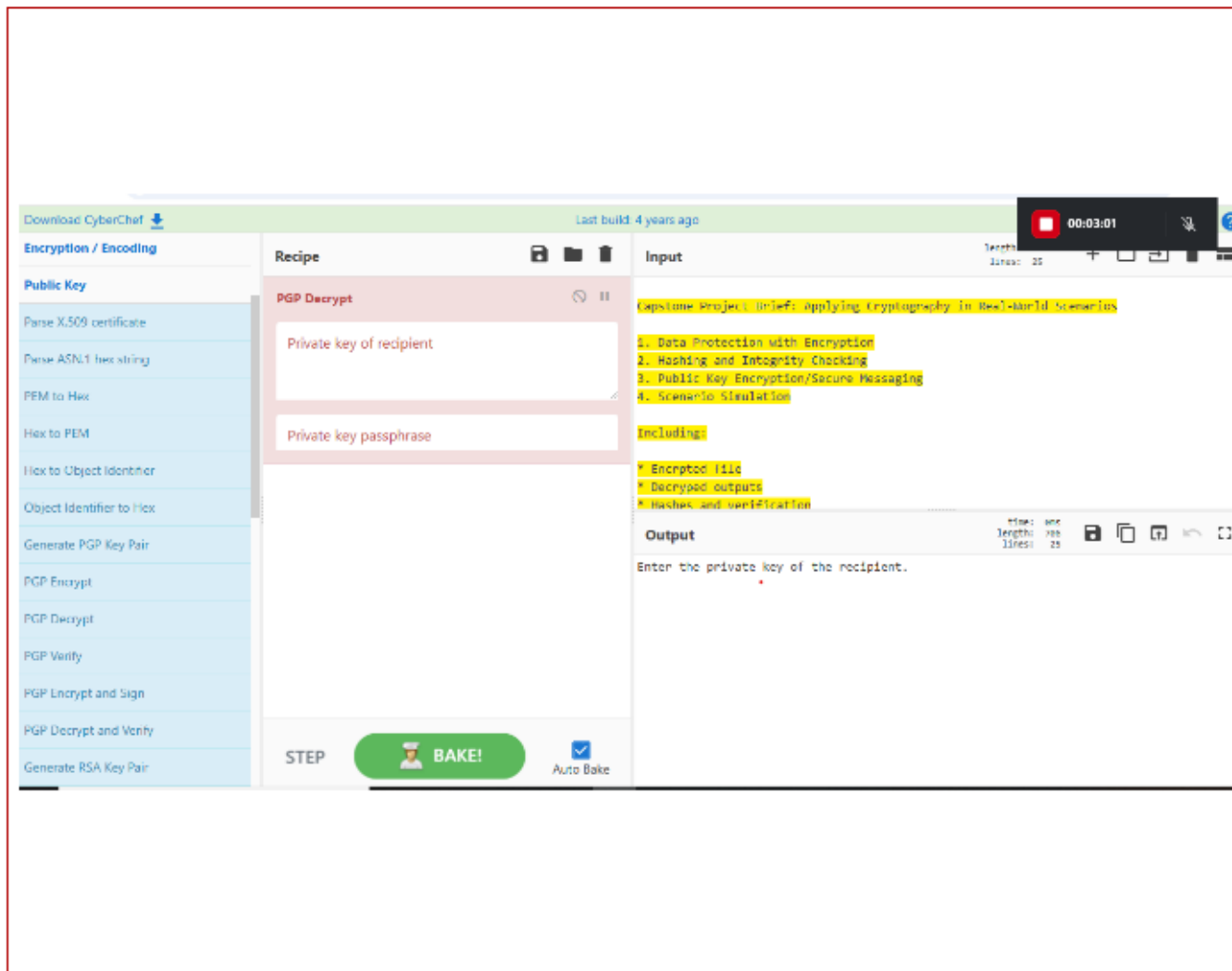
Tools Used

Tool	Use in the Project
GPG / GnuPG	File encryption, decryption, RSA key generation, public key encryption, and digital signature verification.
CyberChef	Browser-based PGP key generation, encryption, and decryption demonstration.
HashCalc	Shows hash values before and after file modification.
SHA-256 command line	Provides stronger integrity verification than MD5 for real-world file checking.



Edit with WPS Office

Symmetric Encryption — Plaintext File



Key Point

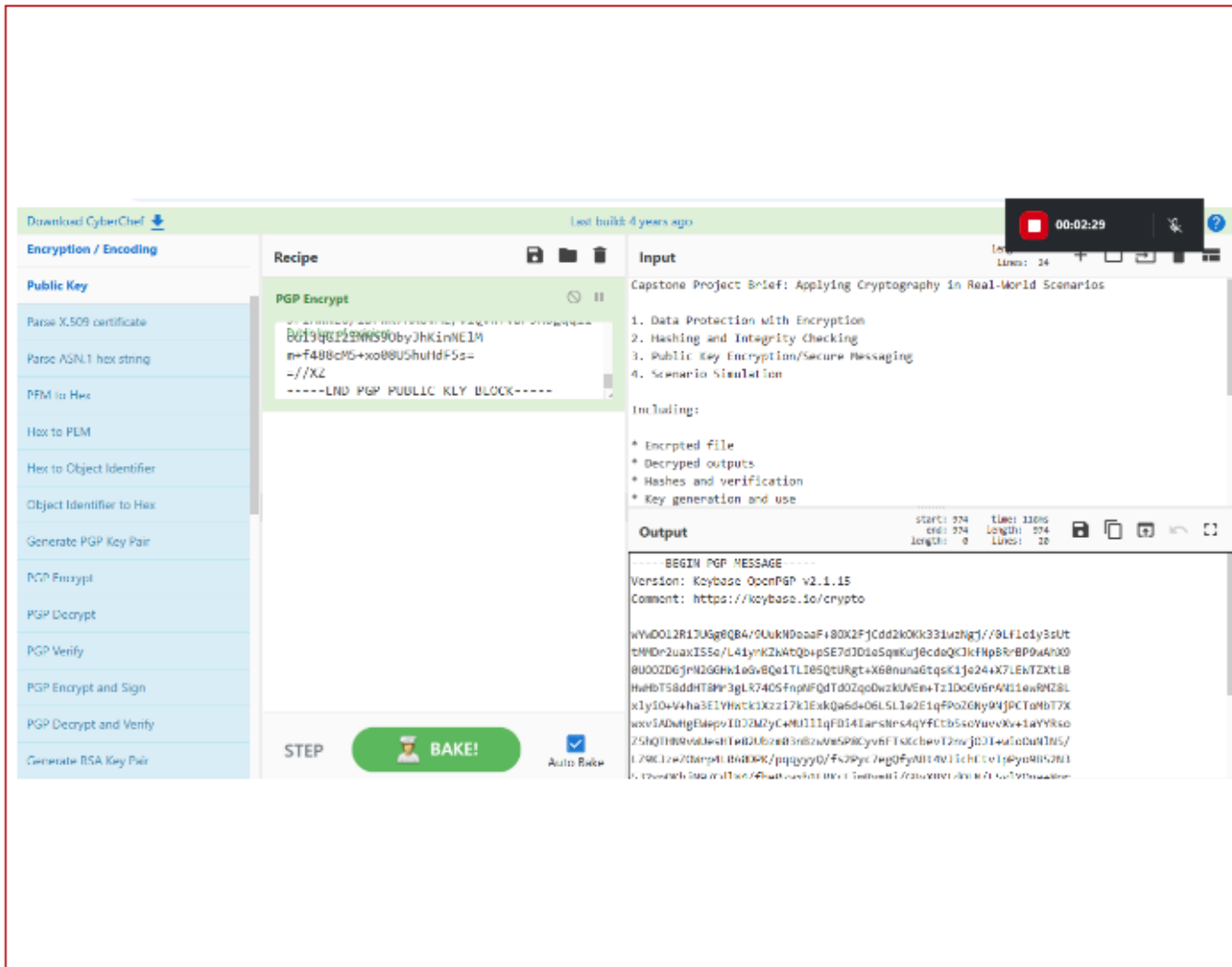
A project sample file was created before encryption.
 The content is fully readable at this stage.
 This is the baseline used for later verification.

Figure 1: Plaintext sample file prepared before encryption.



Edit with WPS Office

Symmetric Encryption — Encrypted Output



The screenshot shows the CyberChef web application interface. On the left, a sidebar lists various tools under the 'Encryption / Encoding' category, with 'PGP Encrypt' selected. The main area is divided into three sections: 'Recipe', 'Input', and 'Output'.

Recipe: A single step named 'PGP Encrypt' is active. The input field contains a PGP public key block starting with '-----BEGIN PGP PUBLIC KEY BLOCK-----'. The 'BAKE!' button is visible at the bottom of the recipe section.

Input: The input text is 'Capstone Project Brief: Applying Cryptography in Real-World Scenarios'. Below it, a list of operations is shown: 1. Data Protection with Encryption, 2. Hashing and Integrity Checking, 3. Public Key Encryption/Secure Messaging, and 4. Scenario Simulation. The 'Including:' section lists: Encrypted file, Decrypted outputs, Hashes and verification, and Key generation and use.

Output: The output section displays the result of the encryption. It starts with '-----BEGIN PGP MESSAGE-----' followed by metadata: 'Version: Keybase OpenPGP v2.1.15' and 'Comment: https://keybase.io/crypto'. The main body of the output is a long, base64-encoded string representing the encrypted data.

Key Point

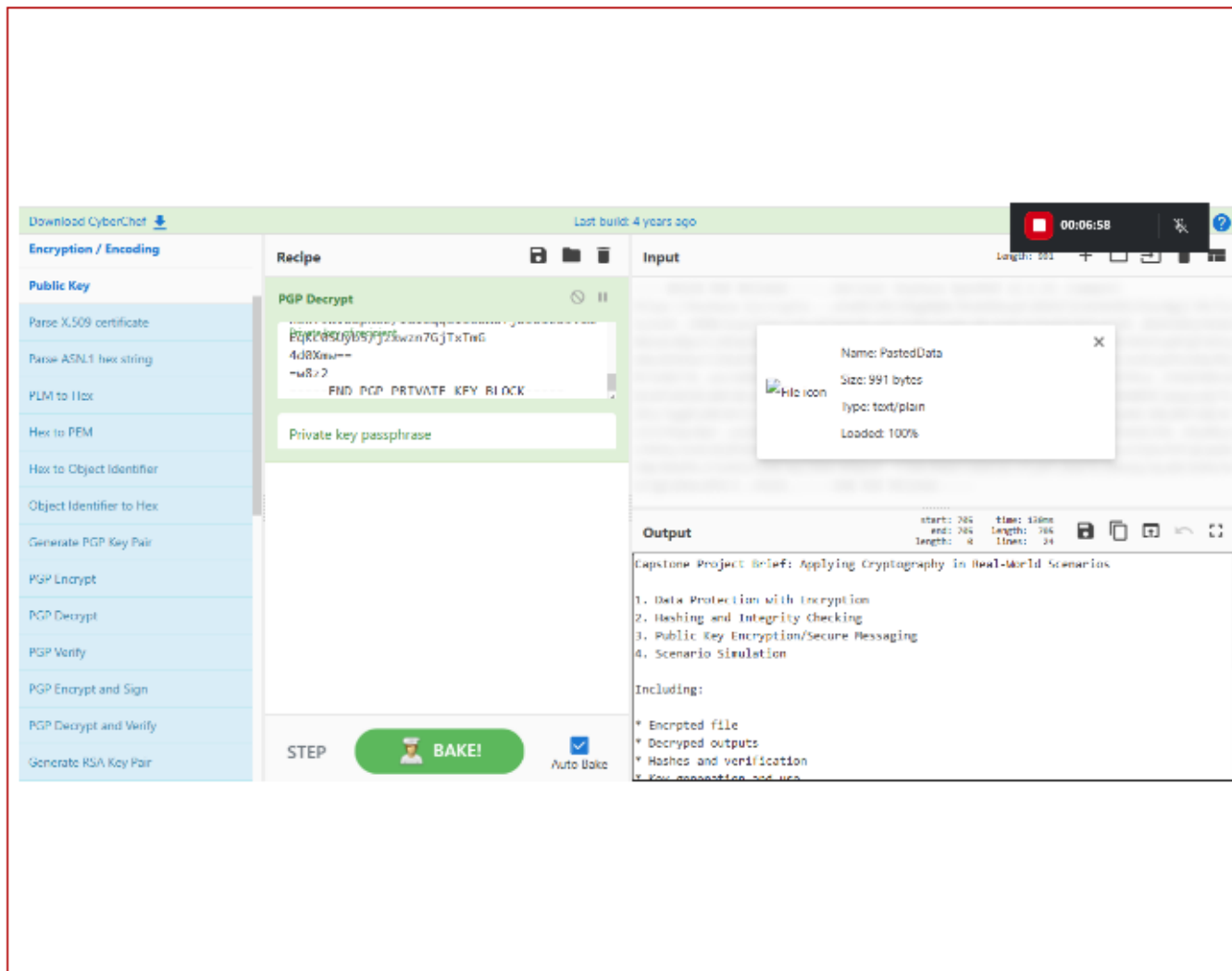
GPG was used with a passphrase to encrypt the file. The output file becomes unreadable ciphertext. This demonstrates confidentiality for stored data.

Figure 2: GPG symmetric encryption command creates encryptionsample.gpg.



Edit with WPS Office

Symmetric Decryption — Restored File



Key Point

The same passphrase was used for decryption.

The restored content matches the original plaintext.

This confirms correct encryption and decryption.

Figure 3: Decrypted output restores the original readable file content.



Edit with WPS Office

Public Key Encryption — Key Pair Generation

```
adeladanseun@seunc: ~/Desktop
Session Actions Edit View Help

adeladanseun@seunc: ~/Desktop
adeladanseun@seunc: ~/Desktop
adeladanseun@seunc: ~/Desktop

(adeladanseun@seunc)~(~/Desktop)
$ gpg --full-generate-key
gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH
This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Please select what kind of key you want:
(1) RSA and RSA
(2) DSA and ElGamal
(3) RSA (sign only)
(4) RSA (sign only)
(5) ECC (sign and encrypt)
(6) ECC (sign only)
(7) ECC (sign and encrypt)
(8) Existing key from card

Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 4096
Requested keysize is 4096 bits
Please specify how long the key should be valid.
0 = key does not expire
<nn> = key expires in n days
<nnw> = key expires in n weeks
<nnm> = key expires in n months
<nnY> = key expires in n years
Key is valid for? (0) 0
Key does not expire at all
Is this correct? (y/n) y

GnuPG needs to construct a user ID to identify your key.

Real name: group6
Email address: tech_crush_group6@gmail.com
Comment: this is a test of asymmetric encryption using built in commands
You selected this USER-ID:
"group6 (this is a test of asymmetric encryption using built in commands) <tech-crush_group6@gmail.com>"

Change (N)ame, (C)omment, (E)mail or (Q)uit? 0
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
```

Key Point

A public key and private key were created with GPG.

The public key can be shared openly.

The private key must remain secret.

Figure 4: GPG RSA public/private key pair generation.

Public Key Encryption — Encrypt to Recipient

The screenshot shows a terminal session where a file named 'encryption sample' is encrypted using GPG. The user prompts for a recipient ID, enters 'group6', and successfully creates the encrypted file 'encrptedsample.gpg'. Subsequently, the user decrypts the file using 'gpg --decrypt encrptedsample.gpg', which outputs the original text of the 'encryption sample' file.

```

adeladanseun@seun: ~/Desktop
Session Actions Edit View Help

$ gpg --encrypt encryption sample
You did not specify a user ID. (you may use *-r*)

Current recipients:
Enter the user ID. End with an empty line: -r
No such user ID.

Current recipients:
Enter the user ID. End with an empty line: group6 (This is a test of asymmetric encryption using built-in commands) <tech_crush_group6@gmail.com>
gpg: checking the trustdb
gpg: marginals needed: 3 completes needed: 1 trust model: gpg
gpg: depth: 0 valid: 1 signed: 0 trust: 0-, 0q, 0n, 0s, 0f, 1u

Current recipients:
rsa4996/F401A8C69A00CDEA 2026-05-31 'group6 (This is a test of asymmetric encryption using built-in commands) <tech-crush_group6@gmail.com>'

Enter the user ID. End with an empty line:
File 'encryptionsample.gpg' exists. Overwrite? (y/N) y

(adeladanseun@seun)~/Desktop $ ls encr*
encryptionsample  encryptionsample.gpg

(adeladanseun@seun)~/Desktop $ cat encryptionsample.gpg
0
000:000:r/h[0N0:0]
0-20 000D2C~00tDt0600y660d666d666;
0N
+P0F0660666_0 00N0B,*0ytK66RGj66-66[0?0
000-0_000r:@0Gt.$j0Uw*00ijw00Uv,00N06|0266c*666Uu6n6666-66667vY0x m)ZT60L00=0.0\NRQ0k0t Vt0 666c+6667660u.66-6
00(-000'4g0065060606S=0N00
p-nI0 00P00Z0 M0PV1000 =00{17.6666's|00c0c0 06601kg082'0'0d0666|00-N06CM
00 :17
00r:000 0L00' 0:000000666660zT20y1,660-10000,100k|000_0J4g000-0F000'U;100k0
j05 +0c10j2000n000260n0m061:666H0y}[E00-60k06666600-0000]0'P0F0;0LjpoX00~000000bn00~J566XV6tr{j66k=:4E0q04E+000}0_0GR000/0
g00
0.'W00vSS0006666|F00q66Hq'0s{0[q066KN k0yk,020-0c000000zQQ000|R00v0Z01,000
(adeladanseun@seun)~/Desktop $ rm encryptionsample
rm: remove regular file 'encrptedsample.gpg'? y

```

Key Point

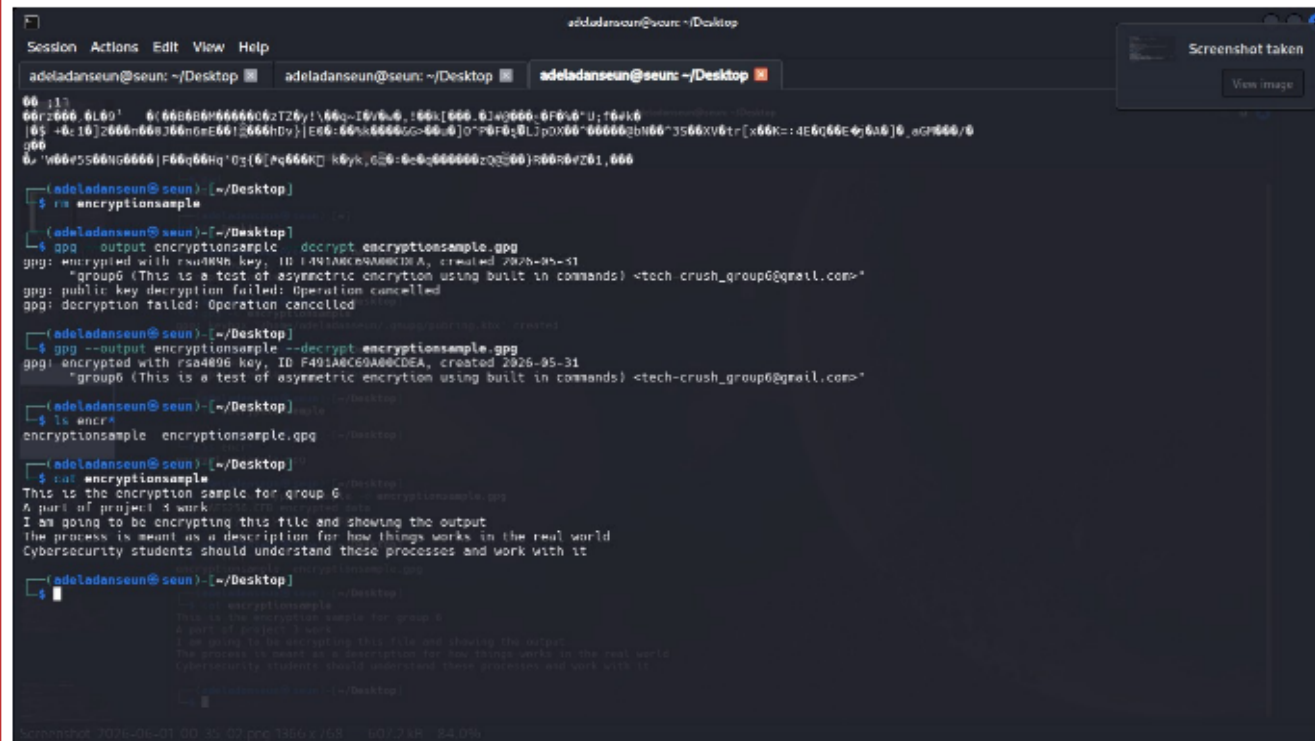
The sender used the recipient public key to encrypt the file. The message can be sent without sharing a secret passphrase first. Only the private key holder can decrypt it.

Figure 5: File encrypted to the selected recipient using the recipient public key.



Edit with WPS Office

Public Key Encryption — Private Key Decryption



Key Point

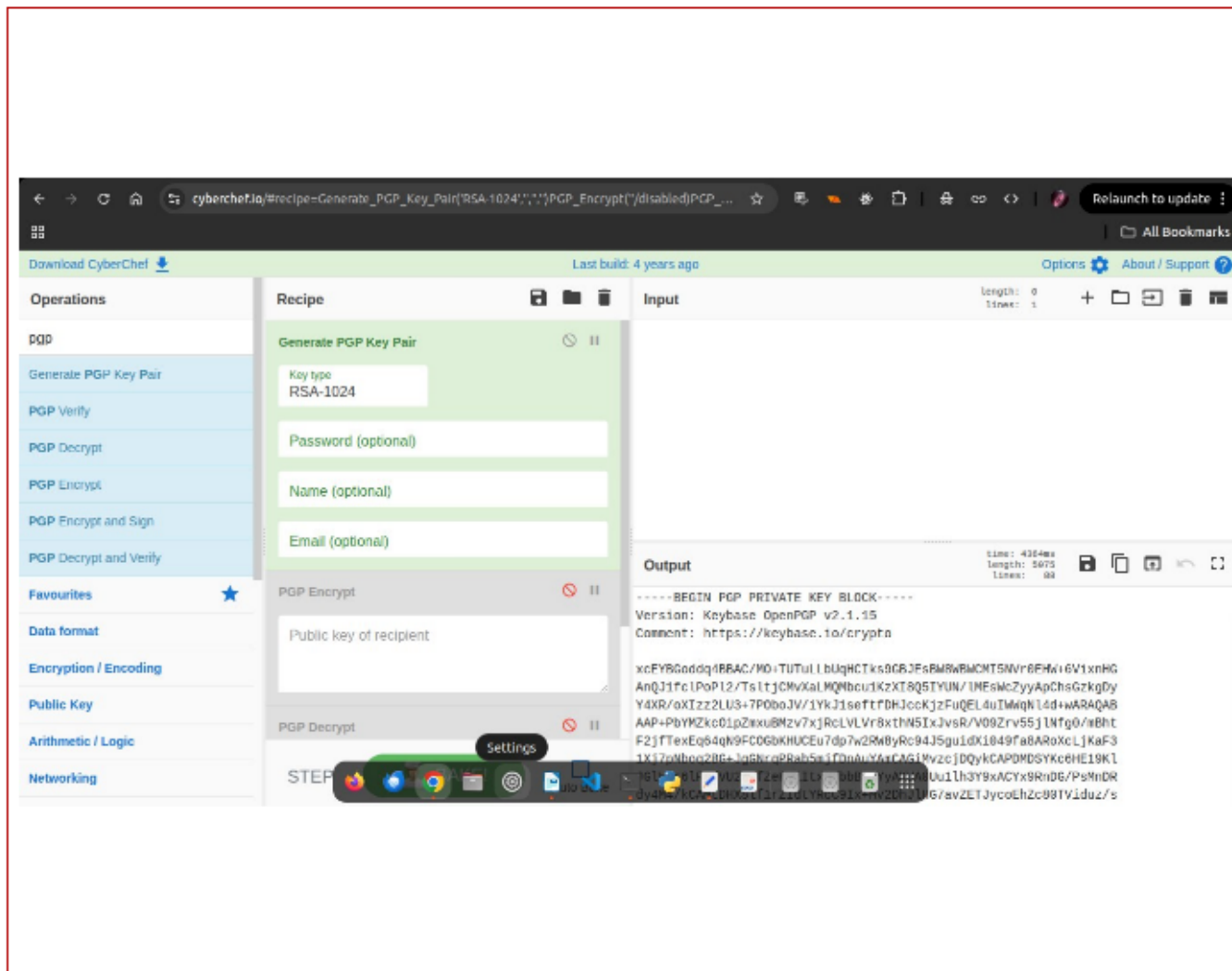
The matching private key was required for decryption. The original file content was recovered successfully. This supports secure messaging and file exchange.

Figure 6: Private key decryption restores the plaintext file.



Edit with WPS Office

CyberChef PGP — Key Pair Generation



Key Point

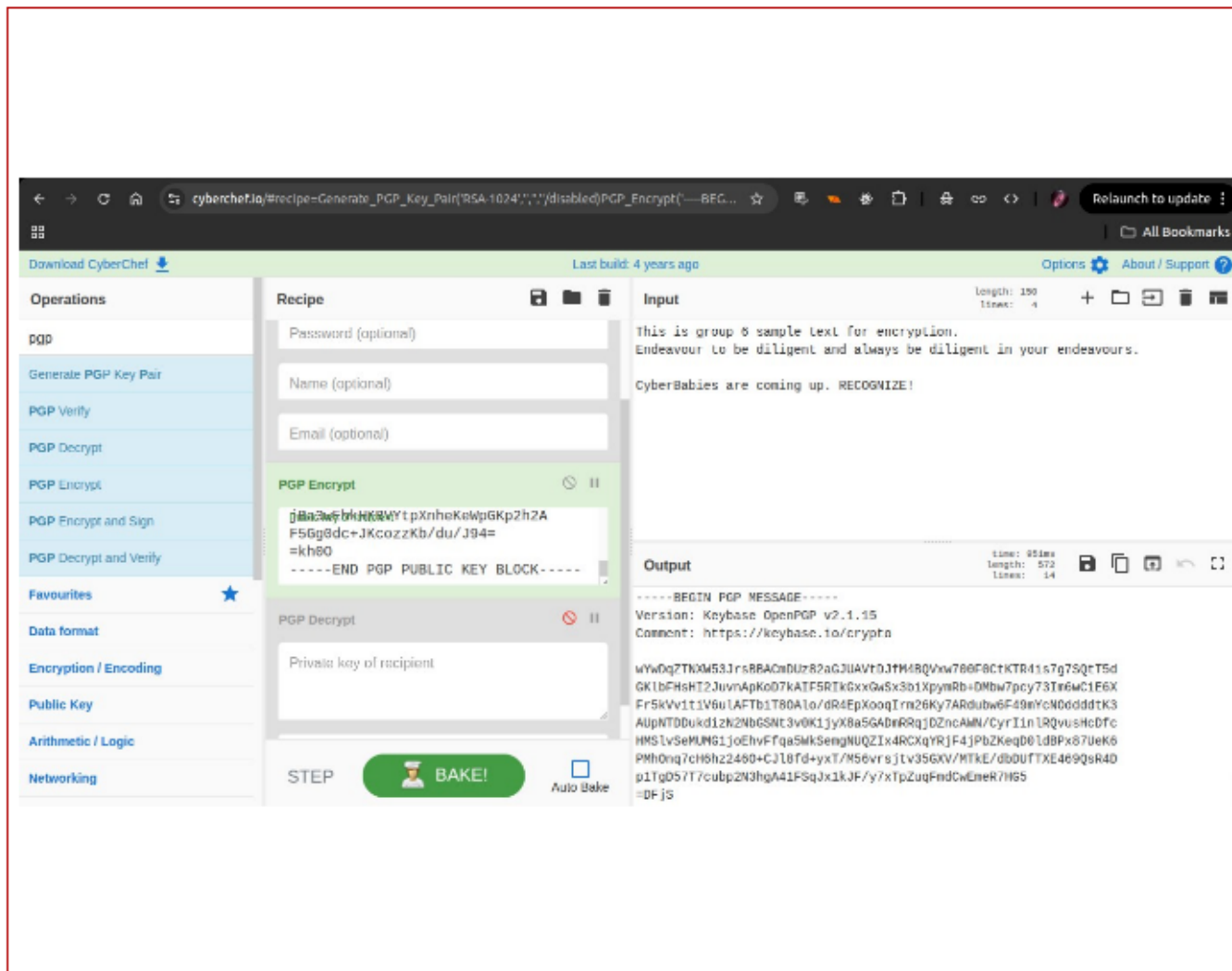
CyberChef visually shows generated public and private keys. It offers a browser-based demonstration of the same concept. This helped confirm the workflow outside the terminal.

Figure 7: CyberChef PGP key pair generation output.



Edit with WPS Office

CyberChef PGP — Encryption



Key Point

The plaintext message was entered in the input panel.

The public key was used to generate the encrypted PGP block. The output cannot be read without the correct private key.

Figure 8: CyberChef PGP encryption turns plaintext into a PGP message block.



Edit with WPS Office

CyberChef PGP — Decryption

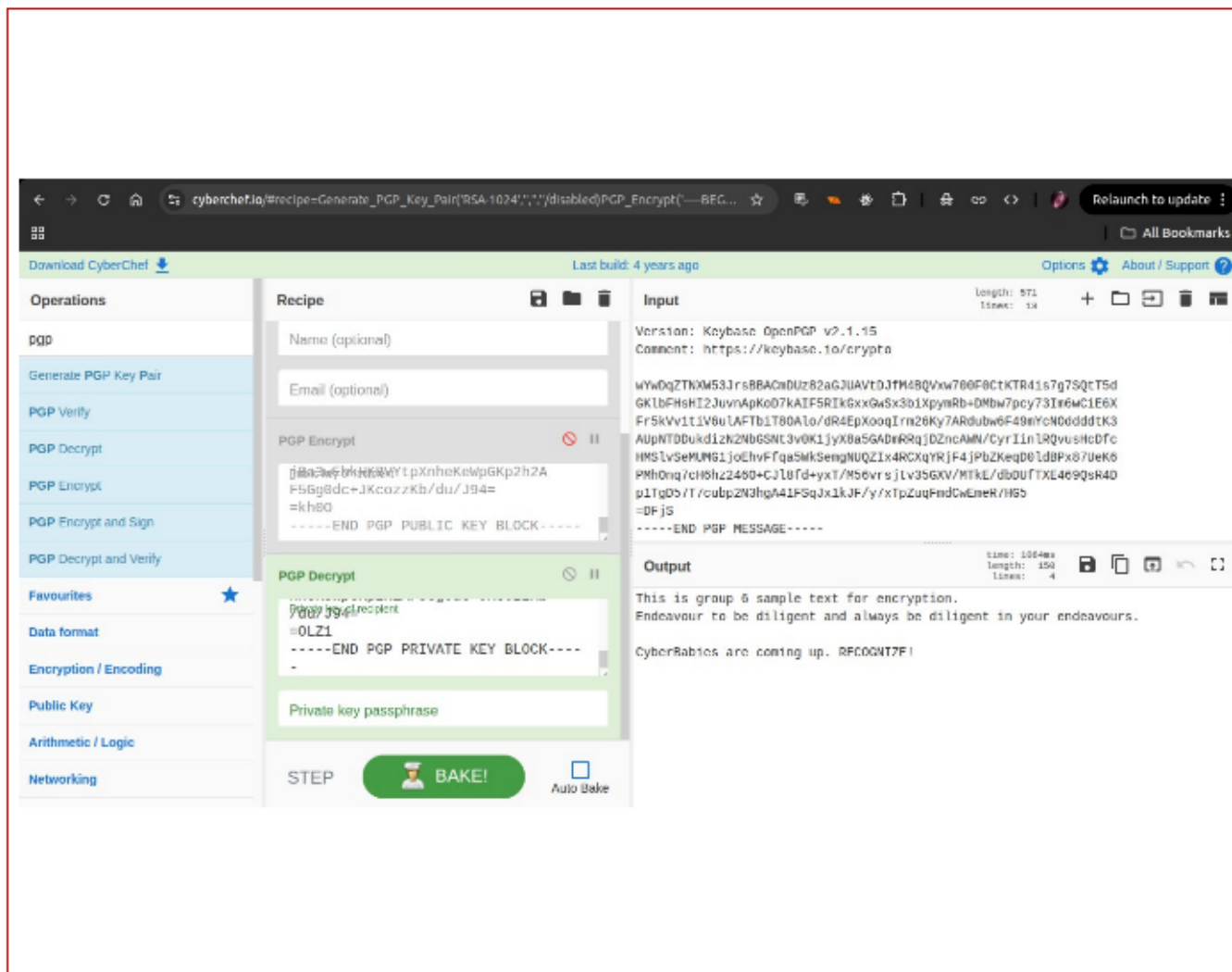


Figure 9: CyberChef PGP decryption returns the original plaintext.

Key Point

The private key was supplied to decrypt the PGP message.

The original readable text was restored.

This confirms the public/private key workflow again.



Edit with WPS Office

Hashing and Integrity Checking — Before Edit

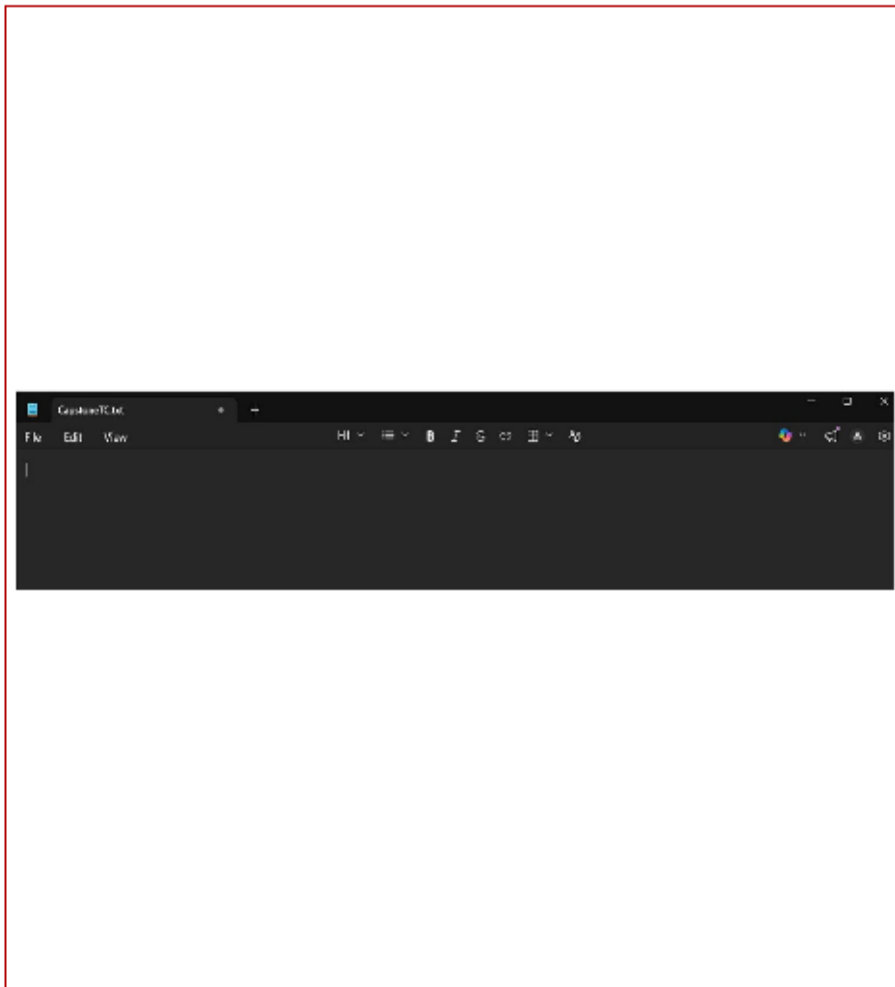


Figure 10: CapstoneTC.txt before editing.



Figure 11: HashCalc output before modification.

The first hash was calculated before any change was made to the file.



Edit with WPS Office

Hashing and Integrity Checking — After Edit

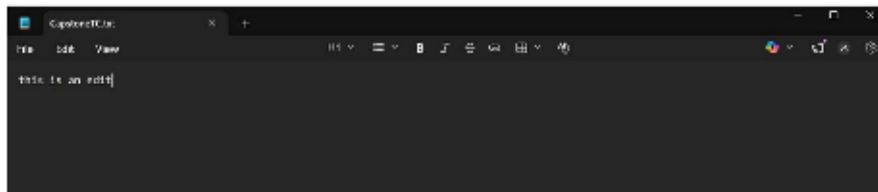


Figure 12: CapstoneTC.txt after editing.

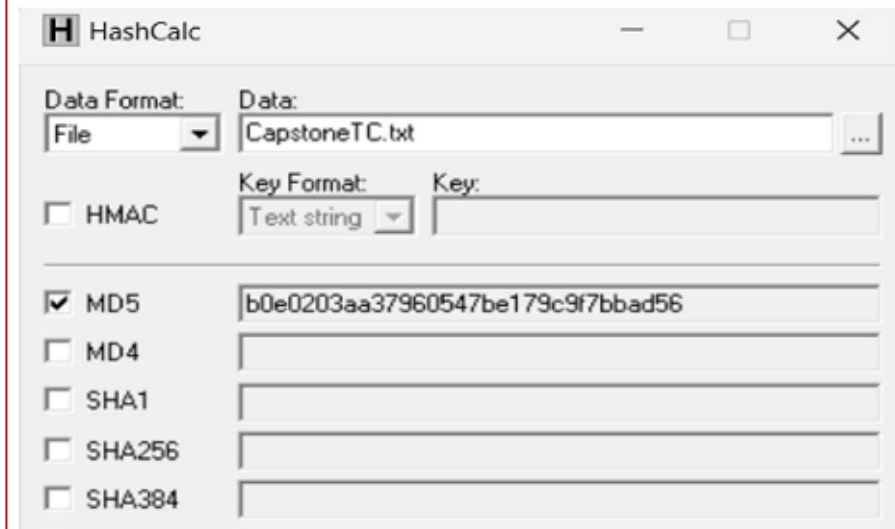


Figure 13: HashCalc output after modification.

After editing the file, the hash value changed. This proves that hashes can detect file modification.



Edit with WPS Office

SHA-256 Integrity Verification

Terminal - SHA-256 integrity verification

```
$ : > CapstoneTC.txt
$ sha256sum CapstoneTC.txt
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855  CapstoneTC.txt
$ printf "this is an edit" > CapstoneTC.txt
$ sha256sum CapstoneTC.txt
7fe4a1571625762a1325f3462bcbee46f19a6848420527c8a2f57d732ffde57c  CapstoneTC.txt

Observation: the SHA-256 digest changes completely after the file content is modified.
```

Key Point

SHA-256 provides stronger integrity checking than MD5.
Different content produced different SHA-256 digests.
This is the preferred approach for real-world verification.

Figure 14: SHA-256 verification showing that the digest changes after file modification.



Edit with WPS Office

Digital Signature Verification

Terminal - Digital signature generation and verification

```
$ export GNUPGHOME=/tmp/project3-gnupg
$ gpg --batch --pinentry-mode loopback --passphrase "" --quick-generate-key 'Group 6 Demo <group6demo@example.com>' rsa2048 sign 0
gpg: Warning: using insecure memory!
gpg: keybox '/tmp/project3-gnupg/pubring.kbx' created
gpg: /tmp/project3-gnupg/trustdb.gpg: trustdb created
gpg: directory '/tmp/project3-gnupg/openpgp-revocs.d' created
gpg: revocation certificate stored as '/tmp/project3-gnupg/openpgp-revocs.d/1E1FA2FE62F5E8C5C3420418C3718393FF517B20.rev'
$ gpg --output project3-message.sig --detach-sign project3-message.txt
gpg: Warning: using insecure memory!
$ gpg --verify project3-message.sig project3-message.txt
gpg: Warning: using insecure memory!
gpg: Signature made Mon Jun  8 22:40:14 2026 UTC
gpg:                using RSA key 1E1FA2FE62F5E8C5C3420418C3718393FF517B20
gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: depth: 0  valid: 1  signed: 0  trust: 0-, 0q, 0n, 0f, 1u
gpg: Good signature from "Group 6 Demo <group6demo@example.com>" [ultimate]
```

Key Point

A detached signature was created using the private key. Verification returned a good signature from the demo key. This proves authenticity and protects integrity.

Figure 15: GPG detached signature generation and verification.



Edit with WPS Office

Results and Findings

Task	Finding
Encryption	Plaintext was converted into unreadable ciphertext.
Decryption	The encrypted file was restored to readable content.
Public key use	A message was encrypted and decrypted successfully.
Hashing	The digest changed after the file content changed.
SHA-256	A stronger integrity check was demonstrated.
Digital signature	The signature was verified successfully.

Main finding: Cryptography protects data only when the correct methods are used and keys are handled properly.



Edit with WPS Office

Conclusion

Project 3 showed how cryptography supports confidentiality, integrity, and authenticity in real-world workflows.

Symmetric encryption protected stored files and backups.
Public key encryption supported secure messaging.
Hashing showed how file changes can be detected.
Digital signatures added proof of origin and integrity.
Key management and careful tool usage remain critical.

The most important lesson from the project is that strong cryptography still depends on responsible key handling and correct implementation.



Edit with WPS Office

References

Group 6 Project 3 Cryptography Final Report.

GPG / GnuPG lab evidence for encryption, decryption, key generation, and signature verification.

CyberChef PGP key generation, encryption, and decryption screenshots.

HashCalc and SHA-256 integrity verification screenshots.

TechCrush capstone presentation template.



Edit with WPS Office

THANK YOU

Group 6 — Project 3



Edit with WPS Office